A LINGUAGEM RoboDIN

A linguagem RoboDIN é uma linguagem de programação experimental destinada ao controle de um robô móvel. A linguagem permite:

- armazenar valores numéricos em variáveis;
- realizar operações aritméticas básicas (`+`, `-`, `*`, `/`);
- movimentar o robô por uma determinada distância por meio do comando `andar`;
- girar o robô por um determinado ângulo por meio do comando `girar`;
- interromper o movimento do robô por meio do comando `parar`.

De modo geral, todo programa na linguagem RoboDIN possui a seguinte estrutura:

```
iniciar robo:
    { <comando> }
finalizar robo.
```

Um `<comando>` pode ser uma atribuição ou um comando de movimentação do robô.

As atribuições possuem a forma:

```
<variavel> <- <expressao>;
```

Uma expressão pode utilizar valores numéricos, variáveis e os operadores aritméticos `+`, `-`, `*` e `/`. Toda variável, assim como toda expressão, é do tipo real.

Os comandos de movimentação possuem as seguintes formas:

```
andar(<valor>);
girar(<valor>);
parar();
```

O argumento `<valor>` pode ser um literal numérico ou uma variável.

Características léxicas

Números

A linguagem só admite literais numéricos não negativos.

Um número pode ser denotado apenas por sua parte inteira ou por uma parte inteira seguida de uma parte decimal com pelo menos uma casa decimal.

Exemplos de números denotados corretamente são:

```
0
10
90
10.2
3.1415
```

Exemplos de números denotados incorretamente são:

```
.20
10.
```

Identificadores

O identificador de uma variável deve ser composto apenas por letras e números, sendo que o primeiro caractere deve obrigatoriamente ser uma letra minúscula. A RoboDIN diferencia letras maiúsculas de minúsculas.

Exemplos de identificadores denotados corretamente são:

```
x
velocidade
velocidadeMedia
distancia1
angulo90
```

Exemplos de identificadores denotados incorretamente:

```
Velocidade
velocidade_media
1distancia
x1_
```

Comentários

A RoboDIN permite comentários de linha única. Um comentário começa com dois caracteres / consecutivos e se estende até o final da linha. Por exemplo:

```
// define a velocidade inicial
velocidade <- 10;
```

O conteúdo dos comentários não deve produzir tokens.

Exemplo de código-fonte correto

O código abaixo corresponde a um programa lexicalmente correto da RoboDIN.

```
iniciar robo:

// define valores iniciais
velocidade <- 5;
tempo <- 10;
angulo <- 90;

// movimenta o robô
distancia <- velocidade * tempo;
andar(distancia);
girar(angulo);
andar(10.5);
parar();

finalizar robo.
```

Exemplo de código-fonte com erros

O código abaixo contém múltiplos erros considerando as definições léxicas da RoboDIN.

```
iniciar Robo:

velocidade_media <- 5;
1tempo <- 10;
distancia <- .5 * tempo;
angulo <- 90.;
andar(distancia = 2);
girar(angulo);

finalizar robo.
```

Instruções de implementação

Antes de implementar o analisador léxico, modele os seguintes aspectos da linguagem:

1. Qual é o alfabeto?
2. Quais são as palavras reservadas?
3. Quais são os tipos de tokens necessários para representar os elementos léxicos?
4. Quais são as expressões regulares para identificar cada token?
5. Quais caracteres podem ser ignorados diretamente pelo analisador léxico?
6. Quais elementos precisam ser reconhecidos pelo analisador, embora não devam gerar tokens?

Após realizar a modelagem, implemente o analisador léxico da RoboDIN utilizando o **PLY**.

O analisador deverá:

- reconhecer corretamente todos os tokens da linguagem;
- diferenciar identificadores de palavras reservadas;
- ignorar espaços e tabulações;
- ignorar comentários;
- manter corretamente o número da linha sendo processada;
- identificar e informar caracteres que não pertençam à linguagem e em quais linhas eles ocorrem.

Além do código do analisador, crie um código principal que processe a entrada e imprima os *tokens* e possíveis erros encontrados.

Utilize diferentes programas RoboDIN para testar seu analisador, incluindo tanto programas lexicalmente corretos quanto entradas contendo erros.

Após o teste do seu analisador, explique porque algumas sequências claramente incorretas, como '`1tempo`', não geraram erro léxico.

Observação: Para utilizar o PLY, basta que a pasta 'ply' esteja na mesma pasta do seu código-fonte. Um arquivo .zip com a pasta 'ply' está disponível no Classroom. Se preferir, você pode baixar o PLY diretamente do github: https://github.com/dabeaz/ply/tree/master